Commerce Intelligence

Developer handover | Deployment Portal | 15 October 2026

Purpose: extend the existing Deployment Portal with a Commerce Intelligence section, develop and validate it on an isolated test URL, then submit a pull request to Sanskar Gupta. Live promotion follows review and explicit release approval; it is not part of this handover.

Area	Verified position / delivery instruction
Live portal	https://deploy-portal.test-magrabi.com - an operational portal, despite the test-magrabi.com hostname.
Current deployment	AKS common/lower cluster; namespace deploy-portal; image :93; one ready replica. Read-only inspection on 15 October 2026.
Local work	Traffic Counts exists locally and is not deployed. Other pending edits include Key Rotation and monitoring/synthetic work. The local snapshot is not an exact copy of the live image.
Repository position	The portal folder is not a working Git checkout on this machine. A canonical main-portal remote was not established. The separate monitoring component has a verified Azure DevOps repository and pipeline.
New test environment	Proposed only: deploy-portal-commerce-test.test-magrabi.com. DNS, certificate, route, namespace, runtime identities and pipeline must be approved/provisioned before use.
Scope of this package	Documentation, existing design/prototype, allowlisted source snapshot and dependency files. No credentials, production data export, deployment, role grant or cloud configuration change.

Start with START_HERE.html, then this guide. Open design/dashboard.html for the illustrative interaction model. The source/integration plan and 19-item access checklist are companion documents, not evidence that all integrations are live.

Reading order: 1) current topology and source map; 2) release and isolation constraints; 3) data architecture and access; 4) development, tests and review gates. The ready-to-use continuation prompt is DEVELOPER_PROMPT.txt.

1. Where the portal runs

Live topology verified through Azure and Kubernetes read-only APIs

Component	Location / configuration
Azure subscription	E-commerce / 72bde893-fa26-4129-a31f-648b81a48153; tenant cc9b4a8c-1b70-4347-acd3-80d32f102f2f.
Hosting	aks-ecom-common-uaenorth-001, UAE North; resource group rg-mepu-common-uaenorth-001. Shared lower infrastructure, not a disposable sandbox.
Public HTTPS	Front Door profile frontdoor-mepu-common-uaenorth-001; endpoint ep-mepu-common; route rt-deploy-portal; origin group og-deploy-portal. Custom domain validated; managed certificate; route forwards HttpOnly to the origin.
Main ingress / service	deploy-portal/ing-deploy-portal-001, host deploy-portal.test-magrabi.com, path /. ClusterIP service magrabi-deploy-portal:80 -> portal container:5151.
Main image	crmepucommonuaenorth001.azurecr.io/magrabi-deploy-portal:93. ACR created 30 Sep 2026 04:25 UTC; digest recorded in reference/live-evidence.json.
Main capacity / storage	1/1 ready. Requests 100m CPU / 128Mi; limits 500m / 256Mi. deploy-portal-runs PVC: 1Gi, RWO, managed-csi; mounted at /data. These are current settings, not a sizing recommendation for new collectors.
Monitoring component	observability/magrabi-monitoring-sit; 1/1 ready; service:80 -> container:5152. Requests 100m / 128Mi; limits 500m / 384Mi; digest-pinned image.
Other same-host paths	/monitoring, /monitoring-assets and /api/monitoring/v1 route to the monitoring service. /rum(/|$)(.*) routes to alloy-rum:12347. Do not collide with these prefixes.

Request path: browser -> Azure Front Door HTTPS -> shared ingress-nginx -> the selected portal/monitoring service. The Commerce Intelligence UI should be part of the existing React application; data collection stays server-side.

The test portal must have its own hostname and resources. Reusing the current hostname, PVC, session key or privileged identities would not create an isolated test environment.

2. Source structure and responsibilities

The ZIP includes an allowlisted local source snapshot, not a production release artifact

Path under magrabi-deploy-portal/	Role / implementation relevance
backend/app.py	Flask portal: authentication, ADO deployment workflows, variables, logging, insights and local Key Rotation work. Large legacy module; it starts background workers on import.
backend/traffic_counts/	catalog.py source/resource mappings; providers.py bounded Azure/Loki readers; service.py jobs/cache/aggregation; routes.py authenticated API; tests/ offline unit tests.
backend/traffic_counts_cli.py	Reusable report collector independent of app.py. Supports source/date selection and aggregate CSV/JSON export.
backend/monitoring/	Separate monitoring application and readers for logs, metrics, traces, RUM and synthetics; wsgi.py/server.py entry points and independent requirements.txt.
frontend/src/routes/	InsightsPage.tsx is the integration point; other pages include Login, Status, Deployments, Admin, Production, Variables, KeyRotation, Feedback and Logging.
frontend/src/trafficCounts/	TrafficCountsPanel.tsx, styles, model and request handling. Reuse bounded report interaction rather than duplicating it.
frontend/src/monitoring/	MonitoringPage and live traffic, service map, dashboards, traces and other monitoring views.
frontend/src/api/, components/	API client/hooks, shared navigation, role gates and common UI. App.tsx owns route registration. Server-side authorization is still mandatory.
frontend/tests/; package*.json	Node test files; npm manifests and exact dependency lock. React/TypeScript/Vite frontend. Vite builds into ../backend/static.
Dockerfile; Dockerfile.monitoring	Main portal and isolated monitoring images. Separate runtimes and builds share frontend source.
k8s/; keyvault/; stage_plans.py	Existing live-target manifests and names-only Key Rotation plans. Reference only: do not apply these to create the test portal.

SOURCE_TREE.txt lists every included source file. Excluded: .secrets, .env files, .git, node_modules, built static assets, runtime logs, deploy_runs, env_changes, feedback, reports and backups. No remote origin or commit is invented for the main portal snapshot.

3. Build and deployment paths

Separate the main portal release from the Monitoring component release

Main Deployment Portal	Monitoring component
Existing documented release path is manual ACR build plus a scoped image/manifests update. No matching dedicated main-portal build or release pipeline was found in the ECom listings inspected.	Azure DevOps ECom pipeline 165: magrabi-observability. Repository: https://dev.azure.com/MGRDevOps-Ecomm/ECom/_git/magrabi-observability
Build context is magrabi-deploy-portal/ ONLY. Dockerfile builds React using npm ci, copies backend/static into Python 3.13 and serves app:app on 5151.	Definition YAML path is /azure-pipelines.yml. The successful monitoring branch uses staged source under monitoring-sit/. Local authoring file: magrabi-observability/azure-pipelines-monitoring-sit.yml.
Gunicorn configuration in source: one worker, eight threads, timeout 120 seconds. Existing run/job state is not designed for arbitrary multiple workers or replicas.	Last verified successful deployed baseline: run 17349; branch codex/sit-monitoring-20260909; commit 0352a20746e68365baf8c7a77fe617e5a1d00367; completed 15 Sep 2026 07:23:54 UTC.
Current image is :93. Source Dockerfile uses node:20-alpine; monitoring uses node:24-alpine. Standardize a compatible reviewed Node version for the new test pipeline rather than silently changing live builds.	Live digest begins 3a26656a75c8; full value in evidence JSON. Pipeline uses build/test/push, digest-pinned plan, environment approval and scoped update. It is not a general portal deployment pipeline.

Pipeline URL: https://dev.azure.com/MGRDevOps-Ecomm/ECom/_build?definitionId=165. Run evidence: https://dev.azure.com/MGRDevOps-Ecomm/ECom/_build/results?buildId=17349&view=results.

- A new reviewed test-only pipeline is required for the Commerce Intelligence portal. Agree the canonical main-portal repository first, commit a vetted baseline, then work in a feature branch with PR checks.

- Do not modify pipeline 165 or reuse its deployment connection without a scoped design review. Its namespace and branch guard belong to the monitoring workload.

- Both images build shared navigation from the same frontend tree. If shared navigation changes, keep both artifacts compatible; test hard reloads and cross-page navigation.

- Never upload the workspace root as an ACR build context: it contains private operational data. The checked .dockerignore is not a substitute for a restricted context.

4. Safety gates before starting the backend

Do not launch the source snapshot with live credentials

Finding	Required handling in the test implementation
Import-time automation	backend/app.py calls _start_background_refreshers() and _start_orphan_recovery(). One refresher is an ADO late-approval sweeper. Add and test a fail-closed automation-off mode before starting it in test. No existing universal safe-mode flag was verified.
Automatic credential loading	app.py looks for ../../.secrets/azuredevops.env. Extract the handover outside the live operational workspace. Do not copy that folder or permit ambient credential discovery in the test service.
Local/live manifest drift	Local k8s/deployment.yaml references deploy-portal-keyvault, Key Vault configuration and an extra Azure Secret; live deployment has no explicit serviceAccountName and only ado/auth/insights envFrom. Blind apply would enable unrelated changes.
Privileged portal features	Disable deploy/retry/approve/cancel/rebuild, variable writes, credential rotation and approval/recovery workers server-side in the Commerce test mode. Keep production service connections unavailable even if a route guard fails.
Existing viewer role	Do not assume the legacy viewer role is a safe business-dashboard role: existing code/runbook records exceptions for variable visibility. Review authorization and use an explicitly restricted Commerce permission set.
Restart and scaling risk	The existing deployment/job manager has process-local state. Never restart live during active deployments. Do not add HPA or extra workers without durable job state, distributed locking and session/cache review.
Monitoring source drift	Local monitoring files may contain unreleased synthetic changes. Start from a verified deployed baseline for unrelated monitoring, not the whole mixed local folder.

The new dashboard must fail closed on missing configuration and show unavailable/stale data instead of fabricated zeros. Test data should be synthetic or anonymized; production reads require named source-owner authorization and a bounded query budget.

All Azure/AKS setup, role changes, deployments and restarts require the exact proposed command and explicit approval under the standing operational rules. This package does not grant that approval.

5. Isolated test URL and PR workflow

Proposed target names are design choices, not existing resources

Test resource	Proposed isolation
Hostname	deploy-portal-commerce-test.test-magrabi.com; platform owner must approve availability, DNS, certificate and dedicated edge route. Never overwrite rt-deploy-portal.
Namespace / service	deploy-portal-commerce-test; separately named deployment, ClusterIP service, ingress and network policies. Same AKS cluster only after capacity/security review; shared cluster is not a full security boundary.
Image / release	Separate magrabi-deploy-portal-commerce-test image repository or approved prefix, immutable commit/digest; test-only CI/CD connection and approval gate.
Identity / login	Dedicated test service account/workload identity and test portal users. Unique session signing key, host-only cookie settings and no shared live session/cache.
State / credentials	Own PVC/reporting store/cache; own connector secrets and alert destinations. Never mount the live /data volume, ADO write PAT or CI/CD Key Vault identity.
Access scope	Development uses fixtures/UAT first. Approved production-source readers are separately scoped and cannot mutate orders, payments, source databases or deployments.

- 1. Sanskar/repository owner provides or establishes the canonical portal remote. Reconcile the local snapshot with live :93 and select only intended Commerce changes.

- 2. Create a feature branch; add tested automation-off mode, restricted Commerce roles, fixture providers and isolated entry points. Prove startup causes no ADO write calls.

- 3. Submit the test infrastructure/pipeline plan for exact-command approval. Provision only the approved test resources; no production deployment stage in the developer pipeline.

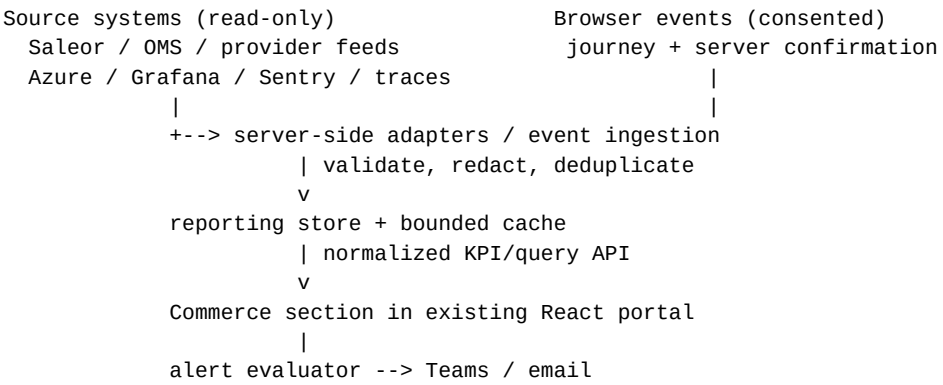
- 4. Implement the dashboard, adapters, reporting store and observability. Use the test URL for reviews and UAT; show source coverage and fixture/live mode prominently.

- 5. Raise a PR to Sanskar with test URL, screenshots, tests, connector scope, schema migrations, load evidence, dependency/security scan and rollback plan.

- 6. After review and explicit live-release approval, promote the reviewed build through a separate controlled live release. Merging code must not silently bypass deployment approvals.

6. Target data architecture

Reuse the portal shell; isolate collection from deployment automation



Dashboard section	Authoritative source and output
Orders / OMS handoff	Saleor order-created records vs OMS parent orders received. Match canonical IDs; show placed, received, lag and missing-handoff exceptions separately.
Payments / methods	Saleor attempt/events plus verified provider outcomes. Tabby, Tamara, Checkout.com, Paymob; method mapping for cards, Apple Pay, KNET and cash/COD.
Traffic / error drill-down	Separate Azure Front Door, ecom-cdn, ECom ingress/Loki and Macula gateway views. Sanitized code -> service -> request/trace context, subject to retained fields.
Infrastructure / APM	Prometheus/Azure CPU, memory, pods/HPA, database and dependency health; Sentry errors; approved trace backend. Production trace coverage must be demonstrated.
Customer journey	Add to Cart -> View Cart -> Proceed to Checkout -> Shipping Accepted -> Payment Details Completed -> Place Order Attempt -> Order Confirmed. Requires new or validated retained events.
Alerts / optional analytics	Threshold and stale-source rules -> approved Teams/email delivery. Clarity/GA4 enrich UX/acquisition; neither replaces live orders or payment truth.

Implement adapters with cursor/checkpoint progress, bounded date windows, pagination, retry/backoff, cancellation and per-source concurrency budgets. Write only to a dashboard-owned reporting store. Do not run full-day production log/DB scans per browser refresh.

7. Access needed: business and payments

Request dedicated source identities; dashboard login alone is insufficient

System / owner	Access and required handoff
OMS / Macula OMS owner + DBA	Named UI viewer for validating totals; supported read-only order/search/export API service identity OR restricted Mongo reporting views. Supply UAT/prod base URLs, schema, pagination, created/updated filters, parent-order rules and Saleor ID mapping. Reference UI route POST /api/v1/order/search is known; service-auth contract is not verified.
Saleor / checkout Backend + DBA	Field-limited reporting API/views for order_order, channel_channel, payment_transactionitem and payment_transactionevent equivalents. TLS/private route, read-only DB role if used, indexed time filters and query budget. Supply brand/country/channel mapping; no blanket admin or DB-write role.
Tabby Merchant + payment owner	Merchant/reporting viewer where supported; merchant/store codes, UAT/live endpoints and verified payment event feed. Approved API reads for reconciliation; default list may omit non-authorized attempts. If no read-only key exists, payment service exposes a constrained reporting facade.
Tamara Merchant + payment owner	Viewer/report export where supported; country/merchant mapping, retained order IDs, read lookup and verified callbacks. Confirm bulk history/backfill. Existing broader API token stays inside payment adapter; do not rotate live tokens merely for reporting.
Checkout.com Account + payment owner	Payment/report viewer, entity and processing-channel IDs, scoped payment-details/report read credential. Confirm scopes against chosen endpoints and entitlement; no capture/refund/void permissions. Reuse verified webhook outcomes.
Paymob Regional merchant owner	Finance/view-only invite, regional endpoint, live/sandbox integration IDs, existing validated callback feed and approved inquiry/export. By-ID lookup is not a complete historical event stream.
Apple Pay / KNET / Cash Payment + OMS owners	Actual provider-to-method mapping per country and recorded payment/COD states. Use existing provider/OMS feeds. No Apple Developer .p8, Sign in with Apple token or B2C credential is required.

Every owner provides: environment, brand/country/legal entity, account/merchant/channel IDs, API documentation, identity and vault reference, available history, rate limits, sanitized response example and person who will reconcile totals. Deliver credentials only through an approved secret manager.

8. Access needed: technical telemetry

Read-only source permissions; provision or instrumentation changes are separate

System / owner	Access, output and feasibility gate
Azure Monitor Azure/DevOps	Dedicated managed identity/service principal with Monitoring Reader or narrower approved metrics scope on selected Front Door/CDN/gateway/DB resources. Supply tenant/subscription/resource IDs. Log queries need separate workspace/table access, retention, diagnostics and quota validation.
ECom Grafana Observability	https://ecom-grafana.test-magrabi.com - dedicated service-account query token, required datasource UIDs, labels and retention. Loki for logs; Prometheus for infrastructure. Viewer scope can be broad by edition: enforce backend query/label allowlists and redaction.
Lower Grafana Observability	https://grafana-lower.test-magrabi.com - separate test/lower read identity and datasource mapping for test validation; it does not establish production coverage.
Macula telemetry Macula DevOps	Approved HTTPS/private Grafana or native log/metric endpoint, dedicated reader, datasource IDs and cluster/service labels. Do not transmit credentials to the previously shared HTTP/IP address.
Sentry Application owner	Organization/project slugs, environment scopes and internal integration read token. event:read and project:read; org:read only if the chosen endpoints require it. A DSN is not an issue-reading token. Verify ingestion/quota coverage.
APM / OpenTelemetry Developers + DevOps	Read access to chosen production trace backend and service names. Separate approved instrumentation PRs, ingestion route/auth, propagation and sampling contract. Existing lower/SIT reader does not prove production traces.
Clarity / GA4 (optional) Analytics + privacy	Clarity project viewer and admin-issued export token; GA4 property Viewer for service identity, property IDs and analytics.readonly authorization. Validate quotas, retention, consent and delay; these are enrichment, not operational truth.

Known Azure resource IDs and deep links are in reference/source-catalog.json and the original access-feasibility.html. WAF logs describe matched rules, not all requests. Exact country/bot breakdown is conditional on actual access/security log coverage; metrics-only access cannot supply it.

Sources: Microsoft Azure built-in Monitor roles; Grafana service accounts; Sentry API permissions. Official reference links are listed in section 14 and in the original access checklist.

9. Access needed: platform, events and alerts

Runtime read access is different from development and provisioning authority

Capability / owner	Minimum practical access and boundary
Repositories / PRs Repo maintainers	Read + branch/PR contribution on the approved portal repository. Separate PR access to storefront, Saleor/checkout and Macula only where event/trace instrumentation is required. Protect main and require Sanskar review.
Test build / deployment DevOps	Test pipeline visibility and queue rights; CI image push to approved ACR repository; scoped deployment identity only for the test workload. Platform owner handles namespace, DNS/Front Door/TLS, identity and role grants after approval.
Portal users Sanskar / platform owner	Dedicated test access and a restricted Commerce role. Live developer access should be limited to approved read-only pages; do not distribute existing shared admin credentials. Review legacy variable-read exceptions before granting viewer.
Secrets / networking Security + DevOps	Dedicated runtime identity can read only its connector secrets in an approved vault. Allowlisted private/TLS routes to APIs, DB, telemetry and outbound alert transport. No subscription Owner, cluster-admin or CI/CD secret-write rights for the collector.
Reporting storage DBA + platform	Dashboard-owned database/schema, writer identity for ingestion, read-only query identity where feasible; backups, retention, restore and migration ownership. Never use a source production DB as the analytics write store.
Journey events Frontend/backend/privacy	Approved schema, consent/pseudonymous journey ID, allowed origins, protected ingestion and server-confirmed order joins. Browser receives no provider/server secret. Test rate limiting, duplicate/retry handling and invalid-event rejection.
Teams + email M365 / channel owners	Dedicated test destination first. Approved Workflow/notification endpoint, owner and backup co-owner; send-only email transport and approved sender/recipients. No company-chat reading or inbox-reading permission is required.
Business acceptance Commerce + QA	Confirm KPI definitions, country/channel/payment taxonomy, benchmark and threshold owners, business hours, escalation recipients, test accounts and reference totals.

Request read-only production connectors only when required for accepted reporting; default development stays on fixtures/UAT. Send-only alerts and writes to the dashboard's own store are explicit exceptions to source read-only access, not permission to mutate business systems.

10. Metrics and correctness contract

Make misleading numbers impossible by design

- Orders: count unique agreed parent/canonical orders. Show placed in Saleor, received by OMS, cancelled and net orders separately. Do not sum Saleor and OMS orders or treat an OMS delay as absence of storefront orders.

- Comparisons: previous day, approved benchmark and average of previous four same weekdays. Use equal elapsed/completed hours in the chosen timezone. Store timestamps in UTC; display IST/Dubai or selected timezone explicitly.

- History: retain selected reporting range plus at least 28 preceding days for four-week weekday comparison, with ingestion-lag margin. Current traffic collector is seven days per run within a 93-day horizon; source retention may be shorter.

- Countries/currencies: UAE, Saudi Arabia, Egypt, Qatar and Kuwait from approved order/channel mapping. Keep AED, SAR, EGP, QAR and KWD separate unless business approves an FX rule. IP geography is not order country.

- Payments: deduplicate attempts and callbacks by stable IDs; initiated/action-required/authorized/captured/failed/refunded are distinct. Approve the denominator for each rate. HTTP 200 or AUTHORIZATION_ACTION_REQUIRED is not payment success.

- Journey: count defined users/sessions/cohorts across events. Surviving checkout rows cannot reconstruct completed journeys because checkouts may disappear when orders are created. Historical events cannot be invented.

- Traffic: keep main Front Door, separate CDN, ingress, filtered storefront and Macula gateway totals separate. Never add overlapping layers or aggregate rows plus their detail. Macula ResponseStatus groups mean 2xx, not exact 200.

- Bots/customers: separate known monitoring, internal/scripts, declared crawlers, browser-like and unknown traffic. Filtered traffic is a customer-facing proxy, not verified humans. WAF rule matches must be deduplicated before counting requests.

- Errors: final HTTP status and upstream retries are separate. Track GraphQL errors and business outcomes alongside transport codes. Drill-down includes allowlisted method, templated path, status, latency and correlation IDs; never raw tokens, cookies, card data or unrestricted bodies.

- Freshness: show source, time window, timezone, collected time, source watermark, lag, retention and completeness. Missing/capped/sampled data is unavailable or partial, never zero. A green connection test does not prove data completeness.

Suggested normalized records: source identity/brand/channel/country; order and external-order IDs; payment attempt/provider reference; event_time/received_time/updated_at; lifecycle state; currency/amount; request_id/trace_id; schema version. Keep join keys server-side and pseudonymize client-facing identifiers where required.

11. Development setup and dependencies

Use a clean checkout outside the operational workspace

Item	Included file / instruction
Main backend	requirements.txt is the exact inspected backend/requirements.txt copy. Python 3.13 matches the Dockerfile. Flask, requests, gunicorn, bcrypt, PyYAML, psycopg[binary], azure-identity and azure-keyvault-secrets are pinned.
Monitoring dependencies	requirements-monitoring.txt is a separate exact copy. Do not install it over the main environment: Flask/Gunicorn versions differ. Keep a separate venv/container.
Frontend	Use source-snapshot/magrabi-deploy-portal/frontend/package.json and package-lock.json with npm ci. Existing stack: React 19, TypeScript 6, Vite 8. Recommended new test CI baseline: reviewed Node 24, matching monitoring source.
Tooling	Git; Node/npm; Python/venv; approved Linux Docker/CI runtime. Azure CLI/kubectl only for approved read-only inspection or specifically approved setup/deployment. Gunicorn is for the Linux container, not a Windows production server.
New connectors	Additional provider/analytics/storage libraries must be selected, pinned, licensed/security-reviewed and tested in the implementation PR. requirements.txt is the current portal baseline, not proof all future connectors are implemented.

Safe setup sequence: verify the package manifest; establish the clean repo; inspect source and dependency lockfiles; create an isolated Python environment; install the baseline requirements; use npm ci in frontend. Implement and test safe mode before running app.py with any source credentials.

```
# Offline Traffic Counts tests; run from the clean portal folder
python -P -m unittest discover -s backend/traffic_counts/tests -p "test_*.py" -v
node --test frontend/tests/trafficCounts.test.mjs

# Frontend only; no production credentials in VITE_* variables
cd frontend
npm ci
npm run lint
npm run build
```

npm run build writes backend/static and may replace a previous build in that clean checkout. Do not run it in the original shared workspace. frontend/vite.config.ts proxies /api to 127.0.0.1:5151 for local development; use a fixture or proven safe-mode backend first.

12. Implementation sequence and acceptance

A working test environment comes before a production release

Phase	Deliverables and exit evidence
0. Baseline + safety	Canonical repo/commit; exact source selection; automation-off startup tests; no privileged connections; approved test namespace/URL/identity design. Existing portal behaviour unchanged.
1. Test foundation	Dedicated test deployment and login, health checks, own storage/secrets, backend RBAC, fixture providers and no external writes. Confirm live portal has not changed.
2. Orders + telemetry	OMS/Saleor reconciliation and five separate traffic views. Date/time, country/brand filtering and exports with scope/freshness labels. Reconcile a completed day to source totals.
3. Payments	Merchant-scoped adapters and final-outcome state model. Tests for decline, action-required, pending, duplicate callbacks, retries, capture, refund and missing/late provider events.
4. Journey + correlation	Approved frontend/backend instrumentation; join a UAT journey to server-confirmed order, logs, Sentry and traces where available. Validate consent, redaction and fail-open instrumentation.
5. Alerts + reuse	Configurable project/source/KPI/threshold mappings; minimum-volume and business-hours guards; stale-source alerts; dedupe/cooldown/recovery; test Teams/email routing; tenant and brand isolation.
6. Review + promotion	PR to Sanskar with test URL, evidence and rollback. Only approved, reproducible image digest promoted after release approval; migrations/backfills and alerts reviewed separately.

- Security tests: anonymous/unauthorized access rejected; restricted user cannot call deployment/key/variable APIs; no secrets in bundles, logs or CSV; no cross-brand/project leakage; SSRF/fixed-origin and pagination limits tested.

- Load/reliability tests: concurrent users, long date range, high-cardinality logs, 429/timeout, missing access, stale data, restart recovery and source outage. Prove bounded memory/query load before sizing or HPA decisions.

- Data tests: equal-hour baselines, partial day, timezone boundaries, cancellation/duplicates, currency separation, missing four-week history and exact-code/group reconciliation.

- UI tests: desktop/mobile, source drill-down and export, keyboard navigation, direct URLs and portal-to-monitoring navigation. Demo values must never masquerade as live results.

13. Live release and rollback checklist

For the later approved release, not an instruction to deploy now

- Record the approved commit, image digest, config/schema versions and expected resource changes. Diff against the current live image and configuration again immediately before release.

- Check the live portal for active deployment runs through approved read-only mechanisms. If visibility is unavailable, stop; do not infer inactivity. A portal restart can interrupt its deployment workers.

- Confirm current capacity, independent backups, reporting-store migration reversibility, session handling and least-privilege identities. Do not copy local Key Rotation/synthetic work into the release unintentionally.

- Present exact Azure/Kubernetes or pipeline actions for approval. Keep namespace, resource names and image digest explicit. Use the existing change/release approvals; no blanket portal or cluster access.

- Deploy only the reviewed changes. Verify readiness, login, permissions, old portal functions, new data-source checks, source reconciliation, navigation across both frontend builds and healthy collector resource use.

- Production alerts start with agreed destinations and thresholds; test destinations never silently become production notifications. All alert rules have an owner, cooldown and recovery action.

- Rollback must restore the captured previous image/config, not an arbitrary previous revision. Pause only the new collectors/alerts through their approved controls; preserve checkpoints and reporting data for reconciliation.

- Database migrations need a separately reviewed rollback/forward-fix plan. Do not delete analytics data or restore a production database as an automatic portal rollback.

PR/review evidence	Required content
Functional	Test URL, branch/commit, screenshots or demo, completed requirements matrix, fixture/live distinction and known limitations.
Operational	Read-only connector grants, query/rate budgets, load tests, source retention/gaps, freshness indicators, dashboards/logs and incident owner.
Release	Image digest, config/secret names only, migration/backup steps, exact target scope, approvals and rollback evidence.

14. Evidence, references and bundle contents

Preserve the distinction between verified state, historical plans and proposed work

File / evidence	Purpose
design/dashboard.html	Original interactive Commerce dashboard prototype; illustrative data, not a live source connection.
design/source-breakdown.html	Complete prior source/integration plan: orders, payments, journey, traffic, APM, alerts, comparisons and ownership.
design/access-feasibility.html	Original 19-item access matrix, feasibility gates and connection diagram; reflects the 14 October review, not newly granted access.
design/access-connection-diagram.png	Existing architecture/access diagram. Additional overview screenshots are included for quick review.
source-snapshot/ + SOURCE_TREE.txt	Allowlisted local portal code, manifests and dependency locks. Local/pending, not live-equivalent or ready to deploy.
reference/live-evidence.json	Sanitized read-only live deployment, routing, pipeline and source-state metadata checked 15 October.
reference/source-catalog.json	Exact source resource IDs and URLs extracted from the non-secret traffic catalog.
requirements*.txt	Exact main and monitoring dependency baselines; use separate Python environments.
DEVELOPER_PROMPT.txt / TEAMS_ACCESS_REQUEST.txt	Continuation instructions and ready-to-send access request. Nothing has been sent externally.
MANIFEST.sha256	SHA-256 for every distributed file except the manifest itself. Verify after extraction.

Current-state evidence: Kubernetes deployment/ingress/service/PVC GETs, Azure account/AKS/AFD/ACR SHOW/LISTs, ADO pipeline definition 165 and build 17349 GETs. Historic release procedure: ENVIRONMENT_RUNBOOK.md, Deployment Portal sections. Local source: Dockerfiles, manifests, app.py, traffic catalog and package lockfiles.

Official references: https://vite.dev/guide/ ; https://learn.microsoft.com/en-us/azure/role-based-access-control/built-in-roles/monitor ; https://grafana.com/docs/grafana/latest/administration/service-accounts/ ; https://docs.sentry.io/api/permissions/ . Provider-specific official links are retained in design/access-feasibility.html.

Open decisions before live connectors: canonical portal repo; approved test URL and capacity; OMS service-auth contract; provider merchant scopes/history; Macula secure telemetry endpoint; production trace coverage; reporting storage; business KPI definitions and alert routing. Source-owner approvals and hosted read tests resolve these, not shared admin passwords.